Extended Essay

Computer Science

An investigation into the use of metaheuristic algorithms to optimize a railway network across Western Europe

RQ: *To what extent can the Genetic Algorithm and Ant Colony Optimization be implemented to design an optimal network of high speed rail lines in Western Europe based on total passenger travel time?*

Word Count: 3996

Introduction **2**
Theoretical background **3**
Metaheuristic Algorithms 3
Genetic Algorithm (GA) 5
Ant Colony Optimization Algorithm (ACO) 7
Problem **9**
Implementation of chosen algorithms **13**
Evaluation function 13
Genetic Algorithm 15
Ant Colony Optimization 16
Results **18**
Analysis **20**
Conclusion **23**
Bibliography **26**
Resources **27**
Appendix **28**
Demand Table 28
Distance Table 28
ACO Code 28
GA Code 34
GA Results 40
ACO Results 41

Introduction

In an era where climate change is a constantly increasing threat to humanity and the demand for sustainable infrastructure is bigger every day, optimization of resources has become an important challenge. Many real-world problems have big search spaces that make deterministic algorithms ineffective.

Metaheuristic algorithms, with their stochastic nature, offer solutions that approximate optima. Their ability to explore vast search spaces whilst also finding good solutions makes them much more efficient in computationally hard problems, in which exact methods struggle (Tomar, Bansal & Singh, 2023).

This paper explores the extent to which the problem of optimizing a high speed rail line network in Western Europe to minimize total passenger travel time can be solved using two important metaheuristic algorithms: Genetic Algorithm and Ant Colony Optimization. By analysing the way in which the algorithms reach their optimum solution, this study aims to evaluate their usability in this problem. The comparison between them is based on the optimized value found and convergence speed.

Thus, the research question for this Extended Essay is: To what extent can the Genetic Algorithm and Ant Colony Optimization be implemented to design an optimal network of high speed rail lines in Western Europe based on total passenger travel time?

To assure that both problems are evaluated equally, a modified Dijkstra's algorithm is used to, based on aeroplane passenger count data, calculate the

total travel time for all passengers. Both algorithms were tested under equal population sizes and generations to further assure a fair evaluation.

Theoretical background

Metaheuristic Algorithms

Metaheuristic algorithms are a class of optimization algorithms that are designed to find an adequate solution to many optimization problems. Such problems usually have an extremely large number of solutions, which makes finding the optimal one through brute force unfeasible or impossible. Metaheuristic algorithms excel in these scenarios as they find good results quickly whilst also not being excessively complex, which is due to two main characteristics: the lack of derivates and their stochastic nature.

Being derivative-free, metaheuristic algorithms do not require these calculations to optimize, making them simpler to implement and more flexible. In many problems, a function to be optimized doesn’t exist, which makes calculus-based approaches unusable. Metaheuristics have the ability to overcome this, as they don’t use derivatives to find good solutions (Tomar, Bansal & Singh, 2023).

The search process is divided into two parts: the discovery phase and the analysis phase. In the first step, the algorithms find the search space’s interesting places and then, in the second step, they carry out local searches in these regions. It is very important to maintain a balance between these two

steps, as focusing on a smaller region, or in other words converging, too fast can make the algorithms prone to not finding the global optimum. This is why their stochastic nature, which implies that the search process starts by generating random results, is very significant. It makes algorithms more likely to look through a bigger range of the search space, increasing the probability of finding the global optimum (Tomar, Bansal & Singh, 2023).

The concept of premature convergence into local optima can be visualized in **Figure 1.** An optimization algorithm may find the local maximum's hill and, with no higher values surrounding it, get stuck. However, this is clearly not the global maximum.

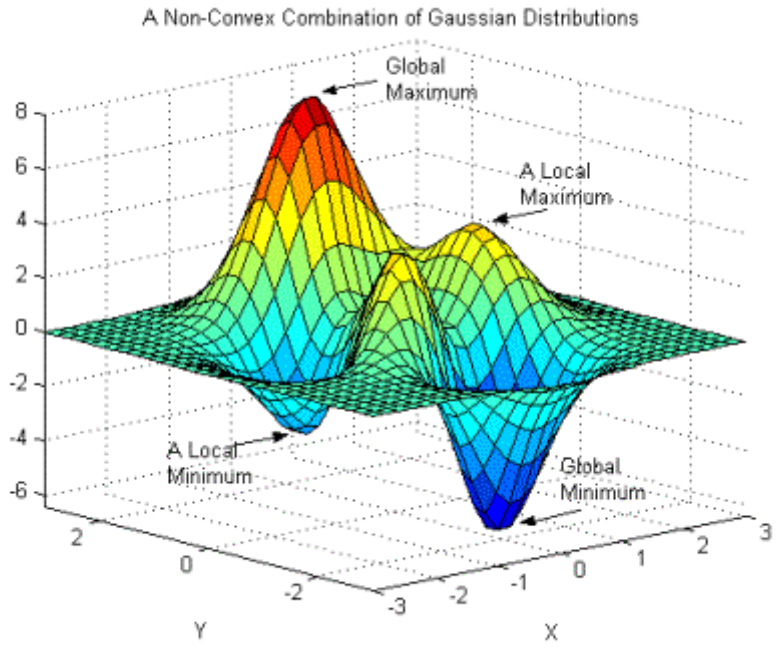


Figure 1: Non-Convex Objective Function

(Kaplan, 2003)

Metaheuristic algorithms are divided in 4 main groups, depending on the origin of their inspiration: human-related, physics-rooted, evolution-based and

based on swarm intellects (Tomar, Bansal & Singh, 2023). In this study, I have chosen to focus on algorithms from the last two categories: genetic algorithm and ant colony optimization.

Genetic Algorithm (GA)

The Genetic Algorithm is a type of metaheuristic algorithm inspired by Charles Darwin's theory of evolution and the concept of natural selection. The algorithm creates an initial "population" and allows it to "evolve" over many generations to find better solutions.

The population of size N is the collection of candidate solutions, each one referred to as an individual, that are being analysed during one given generation. After each evolution cycle, a given number of individuals "die" and new individuals are "born" to replace them. Each individual has its "fitness" score, a value that determines how good the solution is. The way of calculating fitness depends on each problem (Thede, 2004).

After getting the initial population, the process moves to selection, where individuals are chosen to go to the "breeding" phase. There are many methods of selection, but all are based around the concept that the individuals with the highest fitness have a higher probability of being picked. This process is repeated N times and the selected individuals are grouped into pairs. Each pair is mixed to create two new individuals, which go on to make part of the new population. For the breeding process to work correctly, all different solutions must have the same format, so their offspring can be formed correctly. To avoid

stagnancy and prevent the algorithm from converging too soon, each individual has a random chance of mutating, which changes it by a small amount (Thede, 2004).

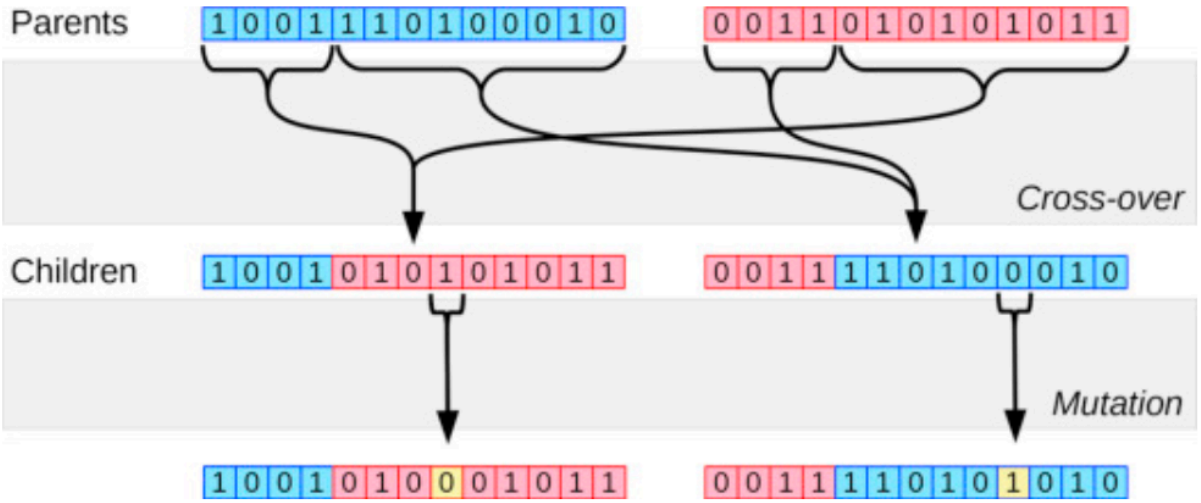


Figure 2: Diagram of a process of crossover and mutation in a Genetic Algorithm **(Anonymous)**

Some versions of genetic algorithms include so-called elitism, where a certain number of the best individuals in a generation are copied directly into the next population, bypassing the cross-over step. This assures that when a new optimum is found, it cannot be lost.

This process is repeated until the answer is satisfactory, which may seem very subjective, and that's because it is. The genetic algorithm, as well as many other metaheuristic algorithms, require parameters and constants to run and for most of them, there is no clear way of getting the best values apart from trial and error. This is due to the fact that each optimization problem has different results and search spaces, which may prefer slower or faster convergence rate. The exceptions to this trial and error are the population size and number of generations. The bigger the population size, the more variance there is in the

candidates, which improves the searching space and increases the probability of the algorithm finding the global optimum. A bigger amount of generations also improve the chances of getting the global optimum. However, both of these are limited by time constraints, and so a balance must be found between them. Returning to the initial question of what is satisfactory, the program is usually designed to end after a certain amount of generations or after the most optimal individual in a certain amount of generations has been the same (Thede, 2004).

Ant Colony Optimization Algorithm (ACO)

The Ant Colony Optimization is a metaheuristic algorithm inspired by the foraging behaviour of ants. When searching for food, they initially explore the area surrounding their nest randomly. As soon as they find food, they return to their nest and, during the journey, drop pheromone trails. When other ants detect this trail, they are likely to follow it. (Dorigo, Birattari & Stutzle, 2006) This process was discovered by and investigated by Denebourg et al. using a similar experiment to the one seen in **Figure 3**.

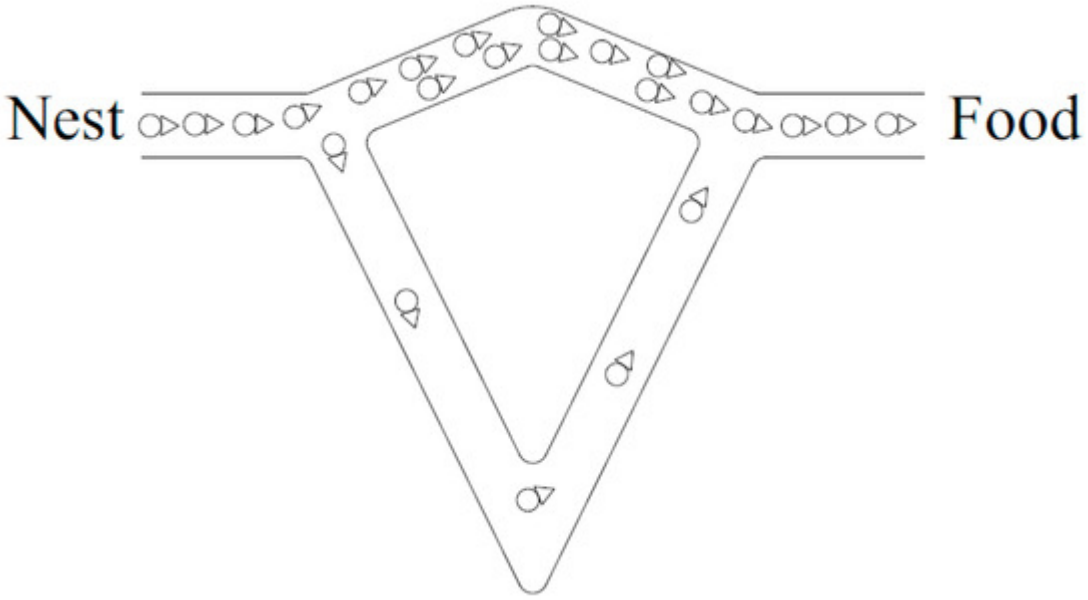


Figure 3: Cooperative search of ants using pheromone trails.

(Ahmid, Dao & Le, 2021)

The ACO algorithm formalizes the natural behaviour of ants into a probabilistic search mechanism. A graph with all connecting nodes is created, with each edge having an equal pheromone value. Each “ant” starts at a node. From there, it decides to go to a neighbouring edge. This process is repeated until some restraints are met, or an objective is found. In the case of a real ant, this could be the food source. As is the case with ant colonies, a lot of ants are generated and do the search process simultaneously.

After all the ants have finished their trajectory, pheromones are deposited along the edges visited by the best ants according to the fitness function. When the next generation of ants begins, when selecting what route to pick, it will no longer be random. Instead, it will depend on the pheromone density of the edges. A possible formula for this probability is the following:

$$P(A) = \frac{Pheromone\ Density\ on\ A}{Sum\ of\ all\ pheromone\ densities}$$

Because ants have a higher probability to follow the edges with more pheromone density, they are enticed to follow previous optimal results, as only they deposit pheromones. This makes it harder for other results to appear, converging very rapidly to an initial solution. To account for this and assure a better analysis of the search space is done, pheromones are “evaporated”. What this means is that after every round, all previous pheromone levels are

decreased proportionally, phasing out old optima for new routes and lowering the convergence speed (Dorigo & Blum, 2005). **Figure 3** shows how a problem might look like after optimization, with most ants following the route with most pheromones but some trying other options.

Problem

During recent times, climate change has pushed society to move to greener solutions for many current industries, such as air transport, that pollute. The more ecological solution to this is high-speed rail (HSR) networks, a technology that's been growing rapidly this century. Many European nations have also implemented their own national systems, but international HSR lines still leave a lot to be desired, as they are limited and expensive (Ey, Bocconi University, & Blue Arches, 2023).

The first step to this green future is the infrastructure; a train cannot go unless it's on rails. However, terrain and other natural obstacles don't allow for many options to be plausible and therefore, there isn't much to optimize (Ey, Bocconi University, & Blue Arches, 2023). For this problem, I have designed a possible way of how HSR infrastructure might look like between Europe's cities with population greater than a million, as seen in **Figure 4**. It is important to point out that this network is only a concept based on current rail tracks (BIL, 2025). For the simplification of the problem, all tracks have a universal speed of 300 km/h, a speed which is already present in various HSR corridors in Europe (BIL, 2025).

This paper was written before the proposition of a more developed European HSR network by the European Comission on November 2025.



Figure 4: Possible future high speed railway network used in the problem

(own work)

Having this set network of rails, the next step is connecting different cities in lines to reach the demand between each pair of locations. The objective is to minimize the total travel time for all passengers, which is expressed in **Function 1**. However, to maintain a certain level of realism in this network of lines, some assumptions and constraints have been put in place.

$$\sum_{0}^{i}\sum_{0}^{j} d_{ij} \cdot t_{ij} \qquad \textbf{(1)}$$

d_{ij} = demand between city i and j

t_{ij} = time between city i and j

The demand values between all pairs of cities have been taken from aeroplane passenger data on Eurostat. Due to the fact that some cities with over a million population used the same airport, locations such as Bruxelles include Antwerp and Lille.

Contrary to aeroplanes, trains have the ability to go through multiple cities, and this is something that must be taken advantage of. For this reason, lines must have 3 stops at minimum and the total amount of lines can be 12 at maximum. Furthermore, to make sure that the lines don't become excessively long, the total time from origin to destination is limited to 6h. Also, to make sure that the optimal network prioritizes direct connections between high demand cities, it is assumed that a transfer between two different lines takes 30 minutes and that trains wait 5 minutes at every stop. It is important to point out that these values are somewhat arbitrary and simplified, and to find the real optimal network other data would need to be used. Finally, if a connection between two cities does not exist, the travel time is counted as 72h, to make sure all cities are in the network.

In this essay, my goal is to explore how the two aforementioned metaheuristic algorithms can be used to solve the HSR line network problem.

Implementation of chosen algorithms

Evaluation function

To make sure the comparison between both metaheuristic algorithms is fair, they both use the same evaluation function based on a modified version of Dijkstra's algorithm.

Both algorithms output a list of 12 or fewer lines, which are lists that contain each city they go through. This data is transformed into a graph containing all connections between cities and which line they're joined by. It is possible for a track between two cities to have multiple lines on it.

From this point, a Dijkstra's algorithm is started from each node until it reaches all other nodes. A dictionary is created with the city as the key and the time it takes to reach that city as the value. In the case that no such possible route exists, the time saved is the unconnected penalty. The possibility of transfers and the fact that these have a time penalty mean that the Dijkstra's algorithm has to be modified.

In the standard implementation of Dijkstra's algorithm, each node stores a single value of the minimum travel time from the origin. The algorithm repeatedly explores neighbouring nodes, updating its values if a shorter route is found. If a node has been reached by a faster route, the value is overwritten

and all its outgoing nodes are reevaluated using its value. This process functions until the shortest path to all nodes has been determined.

However, in the railway network problem proposed in this paper, overwriting can cause incorrect results. This is because the cost of continuing from a node depends on the line used to reach it, due to transfer and stop penalties. Therefore, it is possible that the fastest route to a node doesn't necessarily use the minimum time to reach the earlier one.

This can be seen more clearly in **Figure 5.** In a normal implementation of Dijkstra, where the origin is node A, the shortest time to node B would be saved as the green line's 1h. Then, a transfer would be required to go on the red line and reach C. The total time is therefore:

$$1h \ + \ 30\,min(transfer) \ + \ 2h \ = \ 3h\,30min$$

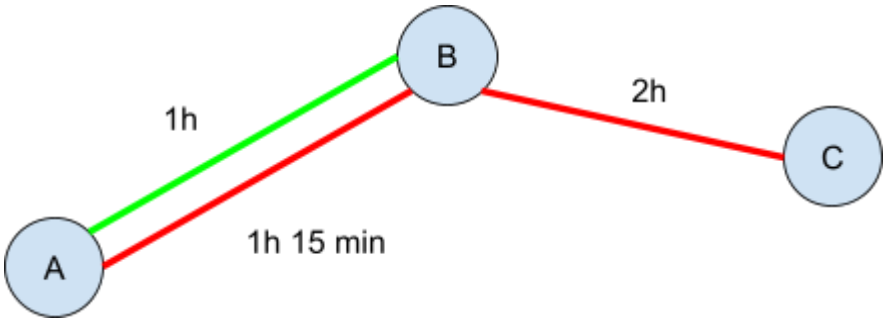


Figure 5: Diagram of 3-node graph connected by two lines

But if we reach B through the red line, even though we would be slower locally, the lack of a transfer means that the total time would be faster:

$$1h\,15min\ +\ 5min(stop)\ +\ 2h\ =\ 3h\,20min$$

To account for this, the modified Dijkstra implementation allows for multiple values at each node, and only overwrites them when the difference is bigger than 25 minutes (transfer time - stop time). When looking to further nodes, the algorithm tries all saved options. This way, it is impossible for a globally shorter route to be lost.

After each value of the minimum time of travel between the origin city and destination has been found, it is multiplied by the value of demand between both cities. The sum of all of these demands is the fitness of a network of lines.

Genetic Algorithm

To generate the initial population N, the genetic algorithm creates 12 random lines that fit the parameters for each individual. This is done by selecting a random city, and from that point creating a route that fits the parameters. To add a bit of variance, there is a probability of lines stopping before achieving the maximum time.

After this, a number of the best results is transmitted directly to the next population, known as elitism. Then, the selection process is done through the tournament selection to get the rest of the population. In this type of selection, a specified amount of candidates from the population are chosen, and the best of these is moved to the breeding pool (Miller & Goldberg, 1995). Due to how the

next step is done, this process is repeated twice as many amounts as the size of the population needed.

The crossover is done by selecting the first x amount of lines from one of the parent lines and the rest from the other parent to create a new candidate. Due to the nature of the problem proposed, it is not necessary to avoid duplicate lines. Finally, when and if a mutation happens, the gene selected is changed by a randomly generated line.

The chosen parameters for this algorithm are specified in **Table 1.**

Population	100
Number of generations	200
Mutation rate	0.2
Tournament size	4
Elite Size	3

Table 1: Genetic Algorithm Parameters

Ant Colony Optimization

Initially, a pheromone graph with the same nodes and edges as the track map is created, with weight 1 on all edges. Then, the initial population is generated. In this implementation, an ant is considered a network of 12 lines. To create each one, a random city is selected, and the following stops are selected using a probability distribution directly proportionate to the pheromone

values on the connecting edges. Due to all values being 1 initially, the first generation is random as the probability to all neighbours is equal.

After this, all candidates are evaluated, and only the most fit solution is used for pheromone depositing. All edges used by the best candidate get the deposit value divided by the fitness of the function summed. After this, the pheromones lose some value determined by the evaporation constant, selected to avoid early convergence while still allowing for pheromone reinforcement.

In the case that the program isn't stagnant (<10 generations from the best ever fitness) the all-time best network deposits pheromone. If it is stagnant (>20 generations), the pheromone values are reset.

To get better results, a specific branch of Ant Colony Optimization called MAX-MIN Ant System has been implemented. This class of ACO is characterized by limiting the maximum and minimum values of pheromones, which effectively limits premature convergence (Stützle & Hoos, 2000).

Number of ants	50
Number of generations	100
Evaporation Constant	0.1
Deposit value	10^8
Maximum Pheromone Value	5.0
Minimum Pheromone Value	0.01

Table 2: Ant Colony Optimization Parameters

Results

The following graphs represent the fitness at each generation after running both algorithms 5 times, a process which took 15 minutes for each algorithm on a 4.2 GHz CPU.

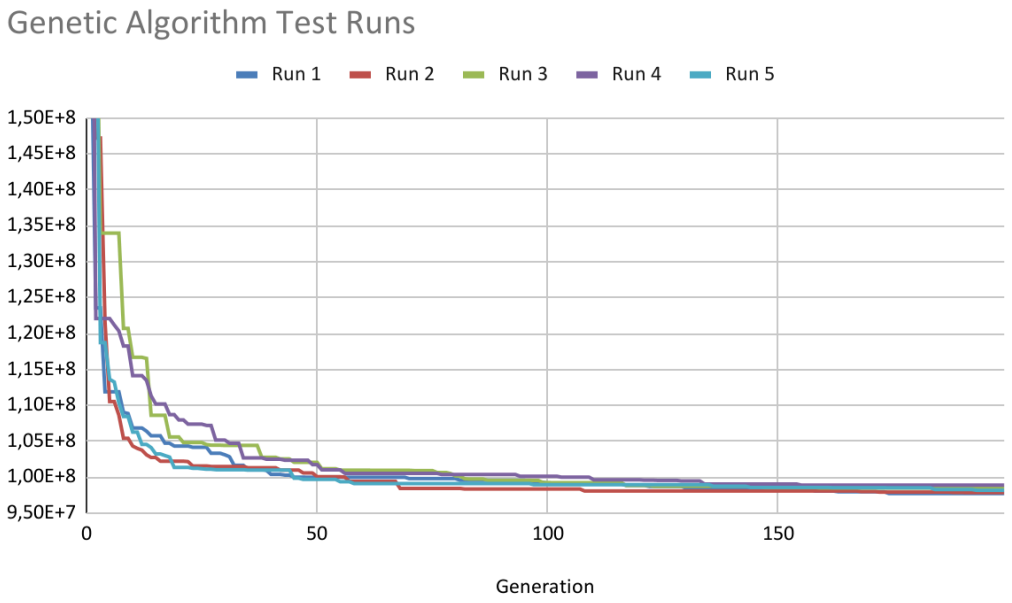


Figure 6: Results from GA runs

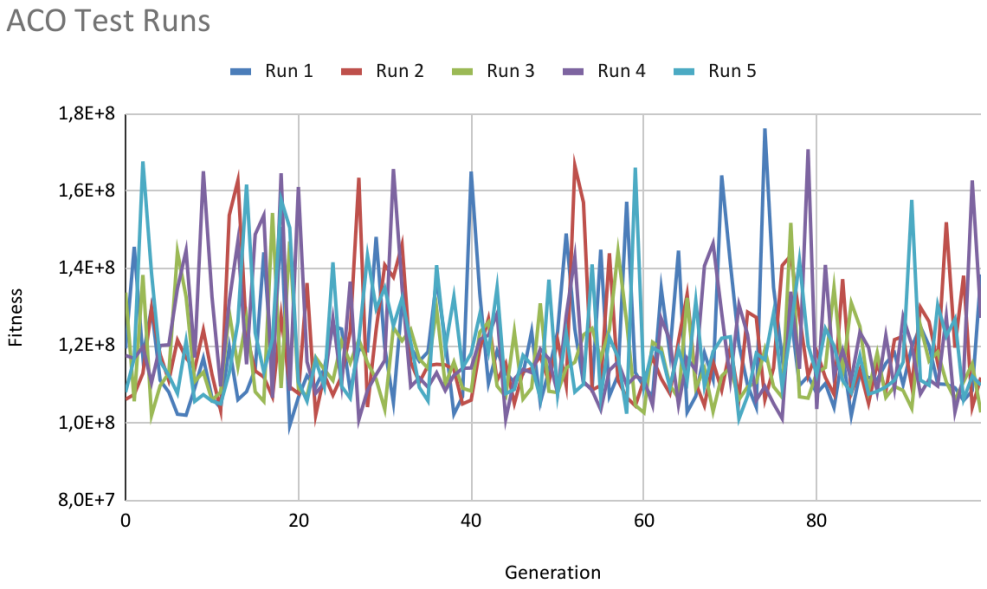


Figure 7: Results from ACO runs

Averaging out the values for all 5 runs of each algorithm, this is how both algorithms compare. It is capped to 100 generations, as ACO doesn't go further.

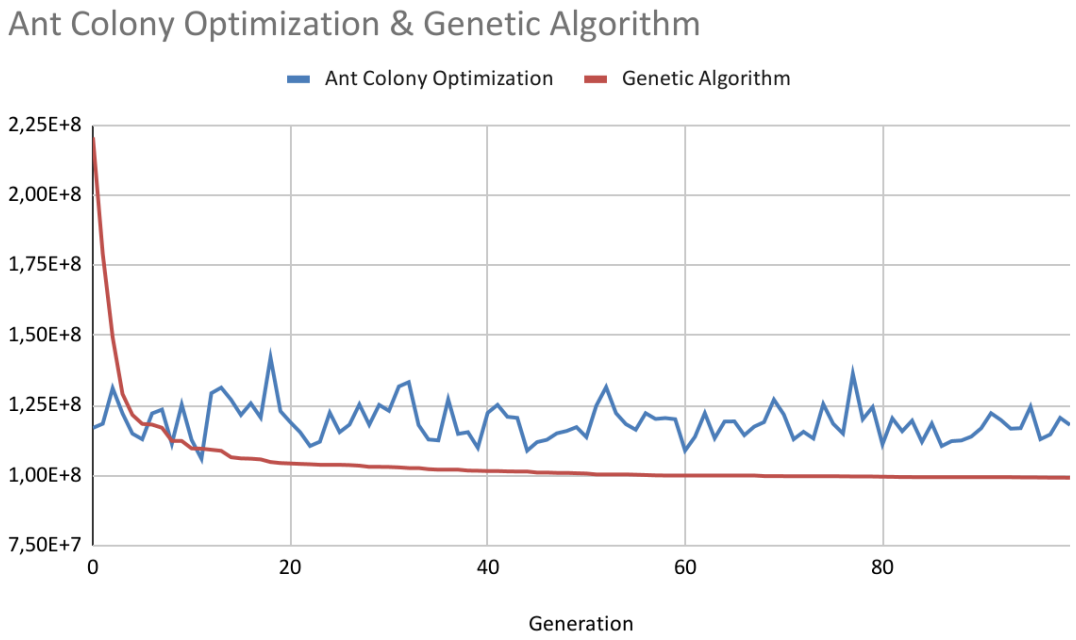


Figure 8: Comparison of ACO and GA

The code for the optimization and the data used is in the Appendix.

Analysis

As is visible in **Figure 6**, the GA consistently gets close to what seems like an optimum. We cannot determine whether this is a global optimum or not due to the nature of metaheuristic algorithms. However, the fact that in all 5 runs the function converges into the same value makes the answer seem good.

The situation is completely different in the ACO implementation. There is no clear optimization pattern, and results seem to not be dependent on previous generations, as seen in **Figure 7**.

There are various possible reasons for the inefficiency of this metaheuristic algorithm in the solution of the problem, but I believe the main one is how the problem in itself is defined and how pheromone depositing works.

When the most fit network is selected, all used edges get pheromones. However, the origin and destination of each line is not saved, nor are the edges used related to the line they were part of in previous generations. What this means is that very little information of actual line structure is passed to next generations. This can be seen in the following example:

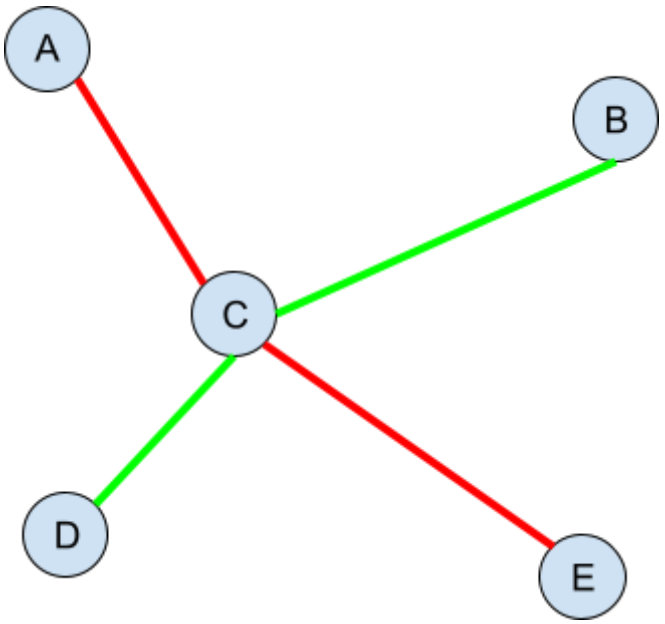


Figure 9: Diagram of 5-node graph connected by two lines.

In a generation of solutions, the most fit candidate is the one represented in **Figure 9**. Therefore, the network deposits pheromones on all the edges used by the lines. The resultant pheromone graph can be seen in **Figure 10.**

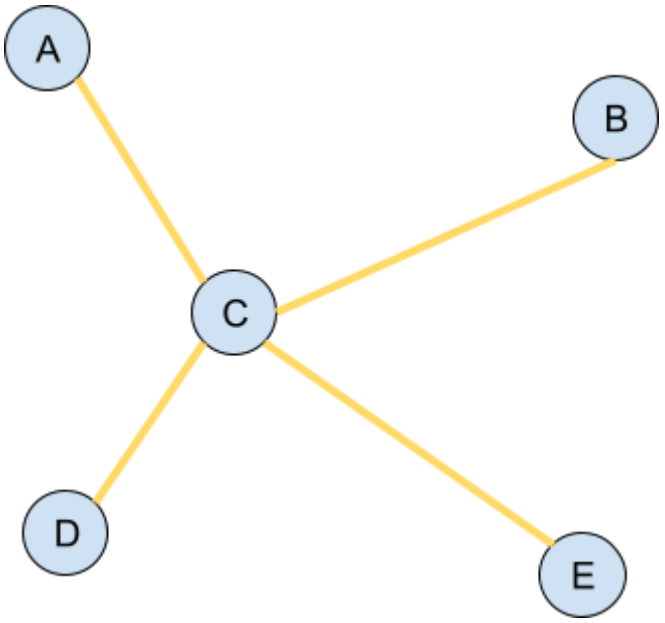


Figure 10: Diagram of 5-node pheromone graph.

When generating the next population, 2 random cities are selected and the pheromone density around them, which in this case is equal at all edges, determines the probability of the selection of the next stop. If, for example, the 2 random origin cities are the C and B, **Figure 11** represents a totally possible solution.

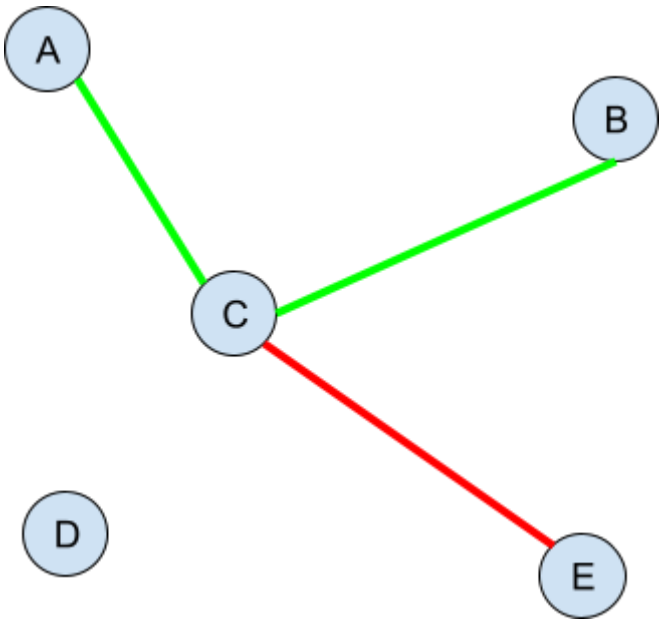


Figure 11: Diagram of 5-node graph connected by two lines after a second generation.

Although the solution proposed in **Figure 11** follows the pheromone trails deposited by the first generation, it is clear that it isn't similar to its predecessor. The weak influence of previous generations makes the results vary a lot and doesn't help the algorithm find the more optimal values found by the GA.

An interesting alternative to this implementation exists. In this algorithm, 12 different graphs, one for each line, would be created for separated pheromone depositing. This would be very helpful in making previous generations' influence stronger. To make it work, origin-destination pairs would have to be chosen, else lines could not be replicated. However, this changes the objective from designing a network to that of finding the shortest routes between 12 pairs of nodes. This would mean that the quality of results would be very dependent on human input and not on the actual algorithms themselves, which is not the goal of this paper.

It is also important to point out that local search methods could be implemented to improve results in general. This, according to Thomas Stützle's paper on the Max-Min Ant System, helps achieve the best results in many problems solved by ACO. This option was considered during the implementation of the algorithm.

$$heuristic = \frac{demand}{distance} \quad (2)$$

As the problem is based on optimizing demand and time, **Function 2** was considered as a good value for local search. However, due to the fact that the demand data is based on aeroplane passenger numbers, this approach would make many good connections unfairly evaluated. A big part of airlines work through a hub and spokes model, where all flights go through a hub airport. What this means in the data is that many city pairs have zero passengers going directly between them, as there are no direct flights. This phenomenon is clearly visible in Germany, where most flights go through Frankfurt (Lufthansa, 2007). Therefore, local search could be useful in this problem, but due to unavailability of actual demand data and the usage of aeroplane passenger counts, it is not.

Conclusion

This investigation set out to explore to what extent the Genetic Algorithm and Ant Colony Optimization can be implemented to design an optimal network of high speed rail lines, using Western Europe as a case study. Both algorithms were evaluated using the same function, making the comparison between both fair.

The results demonstrate that the GA constantly produced more optimal and consistent results. Its optimization process made sure that interesting results didn't get lost and allowed the algorithm to work on refining these solutions. By contrast, the ACO did not manage to converge and consistently gave worse results. The primary reason for this was the weak communication of line structure between generations, demonstrating that pheromones, in the implementation explored in this investigation, did not express networks in a sufficiently good manner.

These findings suggest that the GA is more apt to solve the problem of creating an optimized HSR network.

It is important to acknowledge that this study has several limitations. The usage of aeroplane data to gauge demand between cities is a big approximation. Furthermore, many assumptions have been taken to make the problem more simple: constant train speed, maximum number of lines, constant times for transfers and stops, etc.

Despite these limitations, this investigation gives an insight into how both algorithms work in this problem. Further investigations could explore alternative implementations of ACO, such as the one mentioned during the analysis, to see if it is possible for this algorithm to be useful in the solution of the problem.

Bibliography

1. Tomar, V., Bansal, M., & Singh, P. (2023). Metaheuristic Algorithms for Optimization: A Brief Review. *Engineering Proceedings*, *59*(1), 238. https://doi.org/10.3390/engproc2023059238
2. Thede, S. M. (2004). An introduction to genetic algorithms. Journal of Computing Sciences in Colleges, 20(1), 115–123.
3. *Miller, B.L., & Goldberg, D.E. (1995). Genetic Algorithms, Tournament Selection, and the Effects of Noise. Complex Syst., 9.*
4. Dorigo, M., Birattari, M. and Stutzle, T. (2006) Ant Colony Optimization. IEEE Computational Intelligence Magazine, 1, 28-39. https://doi.org/10.1109/MCI.2006.329691
5. Dorigo, M., & Blum, C. (2005). Ant colony optimization theory: A survey. Theoretical Computer Science, 344(2–3), 243–278. https://doi.org/10.1016/j.tcs.2005.05.020
6. Stützle, T., & Hoos, H. H. (2000). – Ant System. Future Generation Computer Systems, 16(8), 889–914. https://doi.org/10.1016/s0167-739x(00)00043-1
7. Lufthansa Group. (2007). *Sustainability report 2007*. Lufthansa Group. https://www.lufthansagroup.com/media/downloads/en/responsibility/LH-sustainability-report-2007.pdf
8. Ey (Ernst & Young), Bocconi University (Green Centre), & Blue Arches. (2023, January 23). *Smart and affordable rail services in the the EU: A socio-economic and environmental study for high-speed in 2030 and 2050 — Executive Report*. Europe's Rail JU / RailResearch.

https://rail-research.europa.eu/wp-content/uploads/2023/01/HSR_Executive_Report_Final_240123.pdf

Resources

1. Kaplan Z., (2003). Non-convex Objective Function [last visited 6/10/25] https://upload.wikimedia.org/wikipedia/commons/e/e3/Non-Convex_Objective_Function.gif
2. Anonymous. [last visited 6/10/25] https://www.sciencedirect.com/topics/medicine-and-dentistry/genetic-operator
3. Ahmid, A., Dao, T.-M., & Le, N. V. (2021). Enhanced Hyper-Cube Framework Ant Colony Optimization for Combinatorial Optimization Problems. *Algorithms*, *14*(10), 286. https://www.mdpi.com/1999-4893/14/10/286
4. https://ec.europa.eu/eurostat/databrowser/view/urb_lpop1/default/table?lang=en&category=urb.urb_luz
5. BIL, (2025). High Speed Railroad Map of Europe. [last visited 6/10/25] https://en.wikipedia.org/wiki/High-speed_rail_in_Europe#/media/File:High_Speed_Railroad_Map_of_Europe.svg

Appendix

Demand Table

https://docs.google.com/spreadsheets/d/1YgB81EuhmeX2h8uDmBQjOgeHVaBKIrF5/edit?usp=sharing&ouid=102853653659307138593&rtpof=true&sd=true

Distance Table

city_distance_matrix.xlsx

ACO Code

```
    import pandas as pd
    import random
    import numpy as np
    import networkx as nx
    import heapq
    from collections import defaultdict
    from joblib import Parallel, delayed

    # ACO PARAMETERS
    num_ants = 50             # number of ants per generation
    num_generations = 100      # number of generations
    evap_rate = 0.1           # evaporation rate
    pher_dep = 1e8            # pheromone deposit multiplier
    PHEROMONE_MAX = 5.0
    PHEROMONE_MIN = 0.01
    ### CONSTANTS ###

    max_time = 6                #max time for the line (hours)
    min_cities = 3                  #minimum amount of cities in a
line
    speed = 300                 #train speed (km/h)
    stop_time = 5.0/60.0              #minutes spent at each stop
(minutes converted to hours)
    transfer_time = 30.0/60.0    #minutes spent at each transfer
(minutes converted to hours)
    max_lines = 12              #maximum amount of lines
```

```
    unconnected_penalty = 72    #penalty for two cities being not
connected

    ### SPREADSHEETS ###
    adj_matrix     =     pd.read_excel("city_distance_matrix.xlsx",
index_col = 0)  # Distances in between cities
    dem_matrix      =      pd.read_excel("city_demand_matrix.xlsx",
index_col = 0)    # Demand between cities

    cities = list(adj_matrix.index)
    num_cities = len(cities)
    c_to_i = {city: idx for idx, city in enumerate(cities)}

    edges                              =                              []
# Graph of connecting cities
    pheromone  =  np.ones((num_cities,  num_cities),  dtype=float)
# Pheromone matrix

    ## Create a list of all rail tracks
    for i, c1 in enumerate(adj_matrix.index):
       for j, c2 in enumerate(adj_matrix.columns):
           if j <= i:
               continue
           dist = adj_matrix.loc[c1, c2]
           if dist > 0:
               edges.append((c1, c2, dist))




    ######################
    ##### EVALUATION #####
    ######################
    def dijkstra(origin, graph):
       dist = {city: [] for city in graph}

        pq = [(0, origin, [origin], None)] # Priority queue with
time, current city, the path to get there &  what line it's on
       while pq:
           time, city, path, line_id = heapq.heappop(pq)

             if not dist[city] or abs(time - dist[city][0][0]) <=
transfer_time - stop_time:
```

```
                dist[city].append((time, path, line_id))

                #Add all neighbors
                        for neighbor, travel_time, track_line in
graph[city]:
                    new_time = time + travel_time
                    new_path = path + [neighbor]

                    if line_id != None and line_id != track_line:
                        new_time += transfer_time
                    elif line_id == track_line:
                        new_time += stop_time

                            heapq.heappush(pq, (new_time, neighbor,
new_path, track_line))

        return dist



    def evaluate_lines(lines):

        cities = adj_matrix.index
        total = 0
        #Graph of all the routes
        G = {}  #city, travel_time, line_id
        for city in cities:
            G[city] = []
        for line_id, line in enumerate(lines):
            for i in range(len(line)-1):
                c1, c2 = line[i], line[i+1]
                dist = adj_matrix.loc[c1, c2]
                time = dist / speed

                if c1 not in G:
                    G[c1] = []
                if c2 not in G:
                    G[c2] = []

                G[c1].append((c2, time, line_id))
                G[c2].append((c1, time, line_id))

        for origin_city in cities:
```

```
            best_times = dijkstra(origin_city, G)
            for destination in best_times:
                    demand_to_dest  =  dem_matrix.loc[origin_city,
destination]
                possible_routes = best_times[destination]
                if not possible_routes:
                    total += unconnected_penalty * demand_to_dest
                    continue

                min_time = min(possible_routes)[0]
                total += min_time * demand_to_dest

        return total

    # Initialize pheromones on all city pairs

    cities = list(adj_matrix.index)

    def construct_ant():
        lines = []
        for _ in range(max_lines):
            line = []
            start_city = random.choice(cities)
            line.append(start_city)
            total_time = 0

            while True:
                c1 = line[-1]
                # Find neighbors with valid tracks
                neighbors = []
                for next_city in cities:
                    if next_city not in line:
                        dist = adj_matrix.loc[c1, next_city]
                            #Check if there is a track between the
cities
                        if dist > 0 and total_time + dist/speed <=
max_time:
                            neighbors.append((next_city, dist))

                if not neighbors:
                    break  # dead end

                probs = []
```

```
                candidate_cities = []
                for next_city, _ in neighbors:
                    pher = pheromone[c_to_i[c1]][c_to_i[next_city]]
                    probs.append(pher)
                    candidate_cities.append(next_city)
                probs = np.array(probs) / sum(probs)
                c2 = np.random.choice(candidate_cities, p=probs)
                line.append(c2)
                travel_time = adj_matrix.loc[c1, c2] / speed
                total_time += travel_time
                c1 = c2

            if len(line) >= min_cities:
                lines.append(line)
        return lines

    ########################
    ##### MAIN ACO ########
    ########################

    def aco():
        best_network = None
        best_fitness = float('inf')
        fitness_history = []
        for iteration in range(num_generations):
                    ant_networks = [construct_ant() for _ in
range(num_ants)]
            fitnesses = Parallel(n_jobs=-1)(
                    delayed(evaluate_lines)(network) for network in
ant_networks
            )

            iteration_best_idx = np.argmin(fitnesses)
            iteration_best_net = ant_networks[iteration_best_idx]
            iteration_best_fit = fitnesses[iteration_best_idx]

            if iteration_best_fit < best_fitness:
                best_fitness = iteration_best_fit
                best_network = iteration_best_net
                stagnation = 0
            else:
                stagnation += 1
```

```
                print(f"Iteration {iteration+1}, best fitness:
{iteration_best_fit}")

            # Evaporation
            pheromone[:] = pheromone * (1 - evap_rate)

            # Reinforce pheromones for best ant
            for line in iteration_best_net:
                for i in range(len(line)-1):
                    c1 = line[i]
                    c2 = line[i+1]
                    deposit = pher_dep / iteration_best_fit
                    pheromone[c_to_i[c1]][c_to_i[c2]] += deposit
                    pheromone[c_to_i[c2]][c_to_i[c1]] += deposit

            if stagnation < 10:  # Only reinforce if not stagnating
                for line in best_network:
                    for i in range(len(line)-1):
                        c1, c2 = line[i], line[i+1]
                        deposit = (2 * pher_dep) / best_fitness
                                pheromone[c_to_i[c1]][c_to_i[c2]] +=
deposit
                                pheromone[c_to_i[c2]][c_to_i[c1]] +=
deposit

                pheromone[:] = np.clip(pheromone, PHEROMONE_MIN,
PHEROMONE_MAX) #MIN-MAX ACO

            if stagnation > 20:
                pheromone[:] = np.ones((num_cities, num_cities))
                stagnation = 0
            fitness_history.append(iteration_best_fit)
        return fitness_history, best_network, best_fitness

    def run_aco():
        runs = {}
        for run in range (5):
            print(f"RUN {run+1} ########################")
            fitness_history, best_network, best_fitness = aco()
            runs[f"Run {run}"] = (fitness_history)

        df = pd.DataFrame(runs)
        df.index.name = "Generation"
```

```
        df.to_excel("aco_results.xlsx", index=True)
        print("Results saved to 'aco_results.xlsx'.")

    run_aco()
```

GA Code

```
    import pandas as pd
    import random
    import numpy as np
    import networkx as nx
    import heapq
    from collections import defaultdict
    from joblib import Parallel, delayed

    ########################
    ##### DATA READING #####
    ########################


    ### CONSTANTS ###

    max_time = 6 #max time for the line (hours)
    min_cities = 3 #minimum amount of cities in a line
    speed = 300 #train speed (km/h)
    stop_time = 5.0/60.0 #minutes spent at each stop (minutes
converted to hours)
    transfer_time = 30.0/60.0 #minutes spent at each transfer
(minutes converted to hours)
    max_lines = 12 #maximum amount of lines
    unconnected_penalty = 72 #penalty for two cities being not
connected

    ### GENETIC PARAMETERS ###

    POP_SIZE = 100        # number of candidate solutions
    N_GENERATIONS = 200   # iterations
    MUTATION_RATE = 0.3
    TOURNAMENT_SIZE = 4
```

```
    ELITE_POPULATION = 3   # number of elite candidates to carry
over unchanged

    ### SPREADSHEETS ###
    adj_matrix     =     pd.read_excel("city_distance_matrix.xlsx",
index_col = 0) # Distances in between cities
    dem_matrix      =      pd.read_excel("city_demand_matrix.xlsx",
index_col = 0) # Demand between cities


    edges = [] # Graph of connecting cities

    ## Create a list of all rail tracks

    for i, c1 in enumerate(adj_matrix.index):
       for j, c2 in enumerate(adj_matrix.columns):
           if j <= i:
               continue
           dist = adj_matrix.iloc[i, j]
           if dist > 0:
               edges.append((c1, c2, dist))

    ############################
    ### GENERATION ALGORITHM ###
    ############################

    def generate_random_line():

       cities = list(adj_matrix.index)
       start_city = random.choice(cities)
       line = [start_city]
       total_time = 0.0

       while True:
           current_city = line[-1]

           # Find neighbors with valid tracks
           neighbors = []
           for city in cities:
               if city not in line:
                   dist = adj_matrix.loc[current_city, city]
                   #Check if there is a track between the cities
                   if dist > 0:
```

```
                        neighbors.append((city, dist))

            if not neighbors:
                break  # dead end

            # Shuffle neighbors for randomness
            random.shuffle(neighbors)
            extended = False

              #Check if the city can be added to the route (Doesn't
exceed the time limit)
            for city, dist in neighbors:
                travel_time = dist / speed
                stops = (len(line) - 1) * (stop_time)
                new_total_time = total_time + travel_time + stops
                if new_total_time <= max_time:
                    line.append(city)
                    total_time = new_total_time
                    extended = True
                    break

            if not extended:
                break # line has reached maximum size

            # Random chance to stop early
            if len(line) >= min_cities and random.random() < 0.3:
                break

        #Make sure line is not a connection of only 2 cities
        if(len(line) < min_cities):
            return generate_random_line()
        else:
            return line

    def generate_lines():
        lines = []
        for _ in range(max_lines):
            line = generate_random_line()
            lines.append(line)
        return lines

    ######################
    ##### EVALUATION #####
```

```
    ######################

    def dijkstra(origin, graph):
        dist = {city: [] for city in graph}

         pq = [(0, origin, [origin], None)] # Priority queue with
time, current city, the path to get there &  what line it's on
        while pq:
            time, city, path, line_id = heapq.heappop(pq)

              if not dist[city] or abs(time - dist[city][0][0]) <=
transfer_time - stop_time:
                dist[city].append((time, path, line_id))

                #Add all neighbors
                        for  neighbor,  travel_time,  track_line  in
graph[city]:
                    new_time = time + travel_time
                    new_path = path + [neighbor]

                    if line_id != None and line_id != track_line:
                        new_time += transfer_time
                    elif line_id == track_line:
                        new_time += stop_time

                            heapq.heappush(pq,  (new_time,  neighbor,
new_path, track_line))

        return dist



    def evaluate_lines(lines):

        cities = adj_matrix.index
        total = 0
        #Graph of all the routes
        G = {}  #city, travel_time, line_id
        for city in cities:
            G[city] = []
        for line_id, line in enumerate(lines):
            for i in range(len(line)-1):
                c1, c2 = line[i], line[i+1]
```

```
                dist = adj_matrix.loc[c1, c2]
                time = dist / speed

                if c1 not in G:
                    G[c1] = []
                if c2 not in G:
                    G[c2] = []

                G[c1].append((c2, time, line_id))
                G[c2].append((c1, time, line_id))

        for origin_city in cities:
            best_times = dijkstra(origin_city, G)
            for destination in best_times:
                        demand_to_dest  =  dem_matrix.loc[origin_city,
destination]
                possible_routes = best_times[destination]
                if not possible_routes:
                    total += unconnected_penalty * demand_to_dest
                    continue

                min_time = min(possible_routes)[0]
                total += min_time * demand_to_dest

        return total


    ##############################
    ##### GENETIC ALGORITHM ######
    ##############################

    def generate_candidate():
        return [generate_random_line() for _ in range(max_lines)]

    def crossover(parent1, parent2):
        cut = random.randint(1, max_lines-1)
        child = parent1[:cut] + parent2[cut:]
        return child

    def mutate(candidate):
        if random.random() < MUTATION_RATE:
            idx = random.randint(0, len(candidate)-1)
            candidate[idx] = generate_random_line()
```

```
        return candidate

    def tournament_selection(population, fitnesses):
            competitors = random.sample(list(zip(population,
fitnesses)), TOURNAMENT_SIZE)
        return min(competitors, key=lambda x: x[1])[0]



    def ga():
        # Create base population
        population = [generate_lines() for _ in range(POP_SIZE)]
        best_solution = None
        best_fitness = float('inf')
        fitness_history = []

        for gen in range(N_GENERATIONS):
            # Evaluation
            fitnesses = Parallel(n_jobs=-1)(
                        delayed(evaluate_lines)(cand) for cand in
population
            )

            # Tracking
            current_best_idx = np.argmin(fitnesses)
            if fitnesses[current_best_idx] < best_fitness:
                best_fitness = fitnesses[current_best_idx]
                best_solution = population[current_best_idx]
                        print(f"Gen {gen}: best fitness =
{fitnesses[current_best_idx]}")

            # Elitism
            elites = np.argsort(fitnesses)[:ELITE_POPULATION]
            # New population generation with elites included
            new_population = [population[i] for i in elites]
            for _ in range(POP_SIZE - ELITE_POPULATION):
                # Selection
                        parent1 = tournament_selection(population,
fitnesses)
                        parent2 = tournament_selection(population,
fitnesses)
                # Crossover + Mutation
                child = crossover(parent1, parent2)
```

```
                child = mutate(child)
                new_population.append(child)

            population = new_population
            fitness_history.append(fitnesses[current_best_idx])

        # Final best solution
        fitnesses = Parallel(n_jobs=-1)(
            delayed(evaluate_lines)(cand) for cand in population
        )
        best_idx = np.argmin(fitnesses)
        print(f"BEST FITNESS TOTAL: {fitnesses[best_idx]}")
        for line in population[best_idx]:
            print(line)
            print("\n")
                 return    fitness_history,    population[best_idx],
fitnesses[best_idx]


    def run_ga():
        runs = {}
        networks = []
        for run in range (5):
            print(f"RUN {run+1} ########################")
            fitness_history, best_network, best_fitness = ga()
            runs[f"Run {run}"] = (fitness_history)
            networks.append(best_network)

        df = pd.DataFrame(runs)
        df.index.name = "Generation"

        df.to_excel("genetic_algorithm_results.xlsx", index=True)
        print("Results saved to 'genetic_algorithm_results.xlsx'.")

    run_ga()
```

GA Results

genetic_algorithm_results.xlsx

ACO Results

aco_results.xlsx